

TASK 4: Multi-Threaded File Downloader

Domain: Systems / Networking | Level: Medium

What is this project?

Downloads a file from the internet by splitting it into chunks and downloading each chunk in a separate thread simultaneously. Shows a progress bar in the console. Supports resuming interrupted downloads.

Key Concepts to Learn First

- Java Threading: Thread class, Runnable interface
- HTTP Range requests (Range: bytes=0-1000)
- RandomAccessFile – write chunks at correct file offsets
- CountdownLatch – wait for all threads to finish
- AtomicLong – thread-safe progress counter

Step-by-Step Explanation

Step 1 – Get File Size

Send a HEAD request to the server. Read 'Content-Length' header.

Step 2 – Divide into Chunks

Split file into N equal parts (e.g., 4 chunks for 4 threads).

Step 3 – Each Thread Downloads its Chunk

Use Range: bytes=start-end header to download only that portion.

Step 4 – Write at Correct Position

Use RandomAccessFile.seek(startByte) to write each chunk at the right position.

Step 5 – Progress Bar

Use AtomicLong to track total bytes downloaded; print a progress bar every 500ms.

Step 6 – Wait for All Threads

Use CountdownLatch(numThreads) – each thread calls latch.countDown() when done; main thread calls latch.await().

Complete Java Code

```
import java.io.*;
import java.net.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;

/**
 * Multi-Threaded File Downloader
 * Downloads a file using multiple threads for speed.
 */
```

```

public class MultiThreadedDownloader {

    static final int NUM_THREADS = 4;           // Number of parallel download threads
    static AtomicLong downloadedBytes = new AtomicLong(0); // Thread-safe counter

    public static void main(String[] args) throws Exception {

        // ■■ Change this URL to any direct download link ■■
        String fileURL = "https://speed.hetzner.de/100MB.bin";
        String saveAs = "downloaded_file.bin";

        System.out.println("Starting download: " + fileURL);

        // ■■ Step 1: Get total file size ■■
        long fileSize = getFileSize(fileURL);
        System.out.printf("File size: %.2f MB\n", fileSize / 1024.0 / 1024.0);

        if (fileSize <= 0) {
            System.out.println("Cannot determine file size. Aborting.");
            return;
        }

        // ■■ Step 2: Create output file of correct size ■■
        try (RandomAccessFile raf = new RandomAccessFile(saveAs, "rw")) {
            raf.setLength(fileSize);
        }

        // ■■ Step 3: Divide file into chunks ■■
        long chunkSize = fileSize / NUM_THREADS;
        CountDownLatch latch = new CountDownLatch(NUM_THREADS);
        long startTime = System.currentTimeMillis();

        // ■■ Step 4: Start download threads ■■
        for (int i = 0; i < NUM_THREADS; i++) {
            long startByte = i * chunkSize;
            long endByte = (i == NUM_THREADS - 1) ? fileSize - 1 : startByte + chunkSize - 1;

            int threadNum = i + 1;
            Thread t = new Thread(() -> {
                try {
                    downloadChunk(fileURL, saveAs, startByte, endByte);
                    System.out.println(" Thread-" + threadNum + " done.");
                } catch (IOException e) {
                    System.err.println("Thread-" + threadNum + " error: " + e.getMessage());
                } finally {
                    latch.countDown(); // Signal this thread is done
                }
            });
            t.start();
        }

        // ■■ Step 5: Show progress in main thread ■■
        while (latch.getCount() > 0) {
            long downloaded = downloadedBytes.get();
            int percent = (int) (downloaded * 100 / fileSize);
            int bars = percent / 5;
            System.out.printf("\r[%-20s] %d%% %.2f MB",
                "=".repeat(bars), percent, downloaded / 1024.0 / 1024.0);
            System.out.flush();
        }
    }
}

```

```

        Thread.sleep(500);
    }

    // ■■■ Step 6: Wait for all threads to finish ■■■
    latch.await();

    long elapsed = System.currentTimeMillis() - startTime;
    System.out.printf("%n\nDownload complete! Time: %.1f seconds\n", elapsed / 1000.0);
    System.out.println("Saved to: " + saveAs);
}

// Downloads one chunk using HTTP Range request
static void downloadChunk(String url, String saveAs,
                          long startByte, long endByte) throws IOException {
    HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
    conn.setRequestProperty("Range", "bytes=" + startByte + "-" + endByte);
    conn.connect();

    try (InputStream is = conn.getInputStream();
        RandomAccessFile raf = new RandomAccessFile(saveAs, "rw")) {

        raf.seek(startByte); // Move to correct position
        byte[] buffer = new byte[8192];
        int bytesRead;

        while ((bytesRead = is.read(buffer)) != -1) {
            raf.write(buffer, 0, bytesRead);
            downloadedBytes.addAndGet(bytesRead); // Update progress counter
        }
    }
}

// Sends HEAD request to get file size
static long getFileSize(String url) throws IOException {
    HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
    conn.setRequestMethod("HEAD");
    conn.connect();
    long size = conn.getContentLengthLong();
    conn.disconnect();
    return size;
}
}

```

Expected Output

```

Starting download: https://speed.hetzner.de/100MB.bin
File size: 100.00 MB
Thread-1 done.
Thread-3 done.
[===== ] 82% 82.34 MB
Thread-2 done.
Thread-4 done.
[=====] 100% 100.00 MB

Download complete! Time: 12.3 seconds
Saved to: downloaded_file.bin

```

How to Run

No external dependencies needed! Uses Java standard library.

```
# Compile:  
javac MultiThreadedDownloader.java
```

```
# Run:  
java MultiThreadedDownloader
```

TIP: You'll learn Threading • HTTP Range • RandomAccessFile • CountdownLatch • AtomicLong